

# The AI agents

Three AI agents look at a scan's findings and give advice. They **advise only**: `policy.yaml` still decides block, warn or ignore, and nothing the agents say changes a decision or an exit code. They **never see a whole secret**. They are optional: SecureGate works the same without them.

| Agent           | What it says                                                                                                                                                                                                                         |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Triage</b>   | For each blocked or warned finding: likely a real secret, likely a false alarm, or one for a person to look at, how sure it is, why, and the next step.                                                                              |
| <b>Fix</b>      | For a key written into a line of Python or JavaScript: the line rewritten to read an environment variable instead, such as <code>os.environ["ACME_PAY_API_KEY"]</code> , and the import it needs.                                    |
| <b>Incident</b> | For blocked keys: how severe it is, who could see the keys and since when, and the steps to take now, in order. The steps follow SecureGate's own checklist for each kind of key, such as where to revoke an ACME Pay or Stripe key. |

The agents use a model you deploy in **Azure AI Foundry**, Microsoft's service for AI models: `gpt-5.4-mini` by default.

## What is sent to Azure, and what never is

**Sent**: the findings as SecureGate shows them everywhere else (the masked value such as `acme****x9Qz`, the rule, the file and line, the scanners that found it), the policy's rules, commit dates and authors' names for the incident plan, and the lines of code around a finding, **with the value taken out**.

**Never sent**: a whole secret. Before any code leaves the laptop:

1. Each finding's own value is located exactly, by its fingerprint (the same HMAC that made the report), and replaced by `<SECRET>` on every line.
2. Anything else shaped like a key, a random-looking token, a password inside a web address, and every string literal on the finding's line are replaced too.
3. If SecureGate cannot locate the value exactly, for example a private key over several lines, it sends **no lines** of that code at all, only the masked finding.

The tests prove it: on the demo project, with all three agents asking for everything they can, no planted value appears in anything that would be sent to Azure.

**What comes back** is treated as untrusted text: it must fit the agreed format, links, HTML and @ mentions are removed, anything shaped like a secret is hidden as `****`, and a suggested line of code is dropped unless it passes every check. Advice is stored only after it passes the dashboard's checks again.

## Set them up (once)

You need an Azure account and about 15 minutes. Azure for Students needs a school email and no card; a free Azure account needs a phone and a card check.

1. In the [Azure portal](#), create an **Azure AI Foundry** resource (the portal may also call it Azure OpenAI): choose your subscription, a new resource group such as `securegate`, a region that offers `gpt-5.4-mini`, and a name such as `securegate-ai`.
2. In the resource, open **Azure AI Foundry portal**, then **Models + endpoints**, and deploy the base model **gpt-5.4-mini** (deployment type **Global Standard**). Keep its deployment name, `gpt-5.4-mini`.

3. Back in the Azure portal, open the resource's **Keys and Endpoint** page. Copy the **Endpoint**, such as `https://securegate-ai.openai.azure.com`, and **KEY 1**.
4. Tell SecureGate: in the menu, choose **A**, then **4** (or run `securegate ai-setup`). Paste the endpoint, press Enter for the deployment name, and paste the key: it stays hidden while you type. SecureGate saves them in `.securegate\ai.json`; Git ignores that folder, so the key is never committed. Never share that file.
5. Check: in the menu, **A**, then **5** (or `securegate ai-check`, or `make doctor`). It sends one tiny request with no code in it, and prints `PASS AI connection: the model answered.`

Instead of step 4, you can set the environment variables `SECUREGATE_AI_ENDPOINT`, `SECUREGATE_AI_DEPLOYMENT` and `SECUREGATE_AI_KEY`; they come before the file. The merge gate on GitHub uses those.

## Ask them

- **In the menu:** after a scan (choices 1, 2 and 4), SecureGate asks `Ask the AI agents about these findings? [Y/n]`. Or choose **A**, then **1**, any time. **A**, then **2**, shows their advice in the window, and **A**, then **3**, in the dashboard.
- **With a command:** `.venv\Scripts\securegate agents --report findings-demo.json. --only triage,fix` asks only those; `--comment FILE` and `--summary FILE` rewrite the pull request comment and the job summary with the advice in them.

It takes about half a minute. The advice is kept in the report itself, so everything that reads the report shows it:

- the dashboard: the **AI advice** page at the top (the incident plan, the triage and the suggested fixes), an **AI triage** and an **AI fix suggestion** panel on each finding's page, and a line on the overview;
- the pull request comment and the job summary: a section **AI advice: the policy decided, not the AI**;
- the JSON download of the whole report.

## In the merge gate

The merge gate on GitHub asks the agents too, once the repository has the secret `SECUREGATE_AI_KEY` and the variables `SECUREGATE_AI_ENDPOINT` and `SECUREGATE_AI_DEPLOYMENT`: see "The AI agents" in The two gates (merge-gate.pdf). Their advice then appears in the pull request's comment, and in the `findings.json` that `make ci-report` brings to the dashboard. They run after the exit code is saved, so they can never change the check.

## Let the fix agent open a pull request

For a pull request with a key in it, the fix agent can open a second pull request, **into the first one's branch**, that reads each key from an environment variable instead. In the menu: **9**, then **F**, for the last demo pull request. Or:

```
.venv\Scripts\securegate agent-fix --pr 12
```

1. It refuses a folder with unsaved changes, a missing gh login, and a pull request that is closed or comes from a fork.

2. It builds the fix in a temporary folder, from the pull request's branch, so your own files do not change.
3. It scans the pull request's commits on this laptop, so every key is located with this laptop's fingerprint key.
4. The fix agent picks each environment variable's name. **SecureGate makes the edit itself**: the quoted string that holds the key becomes `os.environ["NAME"]` in Python or `process.env.NAME` in JavaScript, and a Python file gets `import os` when it needs it. Nothing else in the file changes. A key that is not alone in its quoted string, such as a password inside a web address, is left for a person.
5. It commits, pushes the one new branch, and opens the pull request. It names each line it changed, with the masked value only.

A key that is only in an older commit, already deleted from the newest code, gets no fix: only revoking it helps. And a fix never undoes a leak: the key is still in the original pull request's history. The new pull request says so, and the incident plan lists the steps.

## If something goes wrong

The findings, their decisions and the exit code never depend on the agents. When an agent cannot answer, the report says so, and everything else stays as it was.

| What you see                                      | What to do                                                                                                    |
|---------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| not set up                                        | Set them up: A, then 4, in the menu.                                                                          |
| Azure refused the key (HTTP 401)                  | The key is wrong or was regenerated: copy KEY 1 again and run the setup again.                                |
| Azure does not know the deployment ... (HTTP 404) | The deployment name or the endpoint is wrong: check <b>Models + endpoints</b> in the Azure AI Foundry portal. |
| Azure is busy or the quota is used up (HTTP 429)  | Wait a minute and ask again, or raise the deployment's quota in the Azure AI Foundry portal.                  |
| could not reach Azure                             | No internet, or a firewall: the rest of SecureGate works offline.                                             |
| The AI advice in this report was withheld         | The advice did not pass SecureGate's checks, so it is not shown. Ask the agents again.                        |

## Limits

- The advice can be wrong. It is labelled as advice everywhere, and the policy's decision stands.
- The agents look at most at 20 findings, with at most 6 tool calls and 8 model calls each, so a run stays small and quick.
- Code is only sent for a report made on this computer, with its fingerprint key. A report from GitHub's merge gate, opened with `make ci-report`, gets advice from the masked findings alone.
- Code in a scanned project can contain text written to mislead an AI. The agents' tools can only read, and only what SecureGate chose to show them, so the worst such text can do is make the advice wrong. More in Limitations (limitations.pdf).